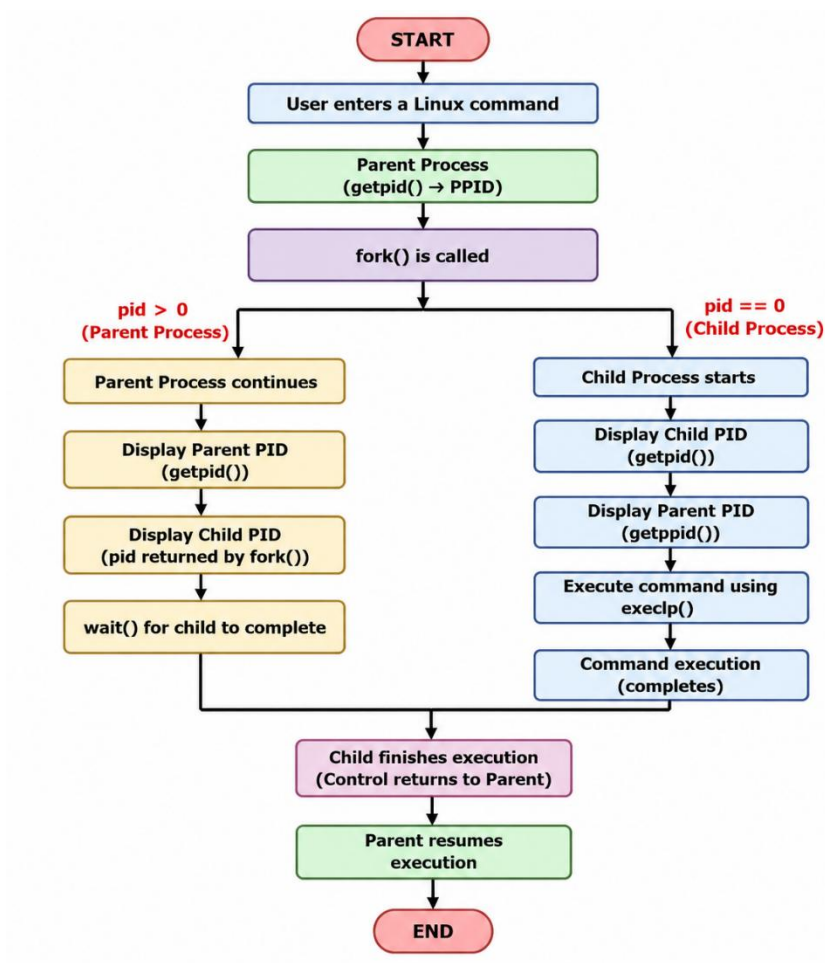


Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

- Write a C code to create a process using fork() system call.
- Read the user input as shell command.
- Use a system call exec() to execute the shell command.
- Use a wait() system call for waiting a parent process.
- Display the PIDs of parent and child processes.



Example Output:

Enter a Linux command: ls

Parent Process

Child Process

Parent PID : 67

Child PID : 68

Child PID : 68

Parent PID : 67

Executing command: ls

CE1 FIRST.c File2.c Proces_States.c command_executor filecopy.c
hello.c process_example session_4_a session_4_b.c

CE1.c File1.c First.c Process_States.c command_executor.c fork_example.c.save my_shell
process_example.c session_4_a.c

FIRST File1.txt Proces_States R1 filecopy hello
os-lab process_example.cyes session_4_b

Child process completed.

```
amrutha@blue:~$ nano command.c
amrutha@blue:~$ gcc command.c -o command
amrutha@blue:~$ ./command
Enter a Linux command: ls
Parent Process
Parent PID : 565
Child PID : 566
Child Process
Parent PID : 565
Child PID : 566
Executing command: ls
InRelease  calc.c  command  file.txt  process.c  test.sh
OSSP       code   command.c  hardlink.txt  pytorch-env  test.txt
calc       code.c  demo.txt  process      shortcut.txt
Child process completed.
```

Thus, the C program successfully creates a child process using `fork()`, executes the Linux command using `execlp()`, and makes the parent process wait for the child using `wait()`.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char command[100];
    pid_t pid;

    printf("Enter a Linux command: ");
    scanf("%s", command);

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("Parent PID : %d\n", getppid());
        printf("Child PID  : %d\n", getpid());

        printf("Executing command: %s\n", command);

        execlp(command, command, (char *)NULL);

        printf("Command execution failed\n");
        exit(1);
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}

```

Task-2: Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

uname identifies the Linux operating system.

lscpu displays CPU architecture, cores, threads, and processor details.

```
amrutha@blue:~$ uname
Linux
amrutha@blue:~$ lscpu
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:               48 bits physical, 48 bits virtual
Byte Order:                  Little Endian
CPU(s):                      12
On-line CPU(s) list:        0-11
Vendor ID:                   AuthenticAMD
Model name:                  AMD Ryzen 7 7445HS w/ Radeon 740M Graphics
CPU family:                  25
Model:                      124
Thread(s) per core:         2
Core(s) per socket:         6
Socket(s):                   1
Stepping:                    0
BogoMIPS:                    6388.08
Flags:                       fpu vme de pse tsc msr pae mce cx8 apic sep mtr
                             r pge mca cmov pat pse36 clflush mmx fxsr sse s
                             se2 ht syscall nx mmxext fxsr_opt pdpe1gb rdtsc
                             p lm constant_tsc rep_good nopl xtopology tsc_r
                             eliable nonstop_tsc cpuid extd_apicid tsc_known
                             _freq pni pclmulqdq ssse3 fma cx16 sse4_1 sse4_
                             2 movbe popcnt aes xsave avx f16c rdrand hyperv
                             isor lahf_lm cmp_legacy svm cr8_legacy abm sse4
                             a misalignsse 3dnowprefetch osvw topoext perfct
                             r_core ssbd ibrs ibpb stibp vmmcall fsgsbase bm
                             il avx2 smep bmi2 erms invpcid avx512f avx512dq
                             rdseed adx smap avx512ifma clflushopt clwb avx
                             512cd sha_ni avx512bw avx512vl xsaveopt xsavec
                             xgetbv1 xsaves avx512_bf16 clzero xsaveerptr ar
                             at npt nrip_save tsc_scale vmcb_clean flushbyas
                             id decodeassists pausefilter pfthreshold avx512
                             vbmi umip avx512_vbmi2 gfni vaes vpclmulqdq avx
                             512_vnni avx512_bitalg avx512_vpopcntdq rdpid f
                             srm
Virtualization features:
Virtualization:              AMD-V
Hypervisor vendor:           Microsoft
Virtualization type:         full
```

ps displays running processes and their PIDs.
top monitors CPU, memory, and process usage.

```
amrutha@blue:~$ ps
  PID TTY          TIME CMD
   311 pts/0        00:00:00 bash
   607 pts/0        00:00:00 ps
amrutha@blue:~$ top
top - 09:50:23 up 30 min,  1 user,  load average: 0.05, 0.04, 0.00
Tasks:  24 total,   1 running, 23 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,   0.0 sy,   0.0 ni,100.0 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st
MiB Mem :  7552.6 total,  6879.2 free,   592.9 used,   230.9 buff/cache
MiB Swap:  2048.0 total,  2048.0 free,    0.0 used,  6959.8 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
    1 root        20   0   24552  15556 11600  S   0.0   0.2   0:00.88 systemd
    2 root        20   0    3180   2212  2072  S   0.0   0.0   0:00.01 init-systemd(Ub
    6 root        20   0    3180   2044   1940  S   0.0   0.0   0:00.00 init
   46 root        19  -1   50424  16800 15528  S   0.0   0.2   0:00.27 systemd-journal
   63 systemd+    20   0   22540  14612 11988  S   0.0   0.2   0:00.10 systemd-resolve
   88 root        20   0   35116  12368  9280  S   0.0   0.2   0:00.34 systemd-udev
  116 root        20   0    2888   2012  1884  S   0.0   0.0   0:00.04 chronyd-starter
  117 root        20   0    4472   3148  2900  S   0.0   0.0   0:00.01 cron
  118 message+    20   0    8816   5384  4724  S   0.0   0.1   0:00.06 dbus-daemon
  148 root        20   0   44092  29836 14692  S   0.0   0.4   0:00.23 networkd-dispat
  165 root        20   0   18416   9512  8280  S   0.0   0.1   0:00.15 systemd-logind
  173 syslog      20   0   220548  6024  4728  S   0.0   0.1   0:00.08 rsyslogd
  229 _chrony      20   0   20420   6428  5652  S   0.0   0.1   0:00.08 chronyd
  235 _chrony      20   0   12092   2524  1884  S   0.0   0.0   0:00.00 chronyd
  284 root        20   0  123456  32648 18036  S   0.0   0.4   0:00.21 unattended-upgr
  288 root        20   0    5492   2832  2644  S   0.0   0.0   0:00.00 agetty
  306 root        20   0    3184   1116   980  S   0.0   0.0   0:00.00 SessionLeader
  307 root        20   0    3200   1252  1108  S   0.0   0.0   0:00.18 Relay(311)
  311 amrutha      20   0    6268   5668  3992  S   0.0   0.1   0:00.10 bash
  313 root        20   0    8648   5572  4728  S   0.0   0.1   0:00.01 login
  373 amrutha     20   0   22332  13564 10952  S   0.0   0.2   0:00.15 systemd
  375 amrutha     20   0   22912   4128  2124  S   0.0   0.1   0:00.00 (sd-pam)
  403 amrutha     20   0    6136   5488  3928  S   0.0   0.1   0:00.06 bash
  617 amrutha     20   0    8164   6108  3964  R   0.0   0.1   0:00.16 top
```

Report:

The OS abstracts hardware resources by managing them through system services. It schedules CPU time, allocates memory to processes, manages storage through the file system, and provides drivers/services for I/O devices.

The operating system acts as an interface between applications and hardware, managing resources efficiently and securely.